



CONFIGURACIÓN DE LA RED EN LINUX Y GESTIÓN DE VLAN

PAR P601



3 DE ABRIL DE 2026
ANDRÉS VIZCAÍNO BAZAGA
DIRIGIDO AL PROFESOR DON ALFREDO ABAD DOMINGO

ÍNDICE

Introducción	2
Comprobación de las herramientas de red.....	3
Instalación de net-tools	3
Identificación de las interfaces de red.....	4
Comprobación de la tabla de enrutamiento	4
Modificación temporal de la dirección IP	5
Modificación temporal de la tabla de enrutamiento	5
Configuración persistente de la red (Netplan).....	6
Gestión de la caché ARP	7
Comprobación de la conectividad: Comando Ping	8
Captura de tráfico de red: tcpdump	9
Auditoría de conexiones con el comando ss	10
Gestión de VLAN en Linux	12
Preparación del entorno virtual en VMware	12
Instalación de las herramientas para VLAN.....	12
Carga del módulo 802.1Q.....	12
Configuración de las VLAN mediante Netplan.....	13
Comprobación de las interfaces y pruebas de conectividad	15
Conclusión	17

Introducción

Para realizar esta práctica, utilizaremos el software de virtualización VMware Workstation Pro, sobre el cual hemos desplegado una máquina virtual con el sistema operativo **Ubuntu Server 24**.

Comprobación de las herramientas de red

Para ello, vamos a escribir en la terminal de nuestro Ubuntu el siguiente comando:

- Ifconfig

```
avb23@avb:~$ ifconfig
Command 'ifconfig' not found, but can be installed with:
sudo apt install net-tools
avb23@avb:~$
```

Como no funciona, debemos de instalar la herramienta net-tools para poder seguir con nuestra práctica.

Instalación de net-tools

Vamos a instalar este paquete desde la terminal. Para ello, seguiremos estos pasos:

1. Primero, vamos a asegurarnos de que la lista de paquetes de nuestro Ubuntu esté actualizada. Escribimos este comando:
 - `sudo apt update`
2. Vamos a instalar el paquete escribiendo lo siguiente:
 - `sudo apt install net-tools`
3. El documento de la práctica también nos recomienda que, por si acaso, instalemos otro paquete llamado ifupdown. Así que, para ir sobre seguro, para instalar ese paquete introducimos el siguiente comando:
 - `sudo apt install ifupdown`

```
No VM guests are running outdated hypervisor (qemu) binaries on this host.
avb23@avb:~$ sudo apt install ifupdown
Leyendo lista de paquetes... Hecho
Creando árbol de dependencias... Hecho
Leyendo la información de estado... Hecho
Paquetes sugeridos:
  apt-transport-https
Se instalarán los siguientes paquetes NUEVOS:
  ifupdown
Se actualizarán 1 nuevos se instalarán 0 para eliminar y 4 no actualizados.
Se necesitan descargar 65,9 kB de archivos.
Se utilizarán 212 kB de espacio de disco adicional después de esta operación.
Get:1 http://mir.archive.ubuntu.com/ubuntu noble/universe amd64 ifupdown amd64 0.8.4ubuntu1 (65,9 kB)
Descargados 65,9 kB en 0s (2.337 kB/s)
Seleccionando el paquete ifupdown previamente no seleccionado.
(Leyendo la base de datos ... 32096 ficheros o directorios instalados actualmente.)
Preparando para desempaquetar .../ifupdown_0.8.4ubuntu1_amd64.deb ...
Desempaquetando ifupdown (0.8.4ubuntu1) ...
Configurando ifupdown (0.8.4ubuntu1) ...
Creating /etc/network/interfaces.
Created symlink /etc/systemd/system/multi-user.target.wants/networking.service → /usr/lib/systemd/system/networking.service.
Created symlink /etc/systemd/system/network-online.target.wants/networking.service → /usr/lib/systemd/system/networking.service.
Procesando disparadores para man-db (2.12.0-4build2) ...
Scanning processes...
Scanning candidates...
Scanning linux images...

Pending kernel upgrade!
Running kernel version:
  6.8.0-71-generic
Dispositivos
The currently running kernel version is not the expected kernel version 6.8.0-107-generic.
Restarting the system to load the new kernel will not be handled automatically, so you should consider rebooting.
Restarting services...
Service restarts being deferred:
/etc/passwd:restart.d/bus.service
systemctl restart systemd-logind.service
systemctl restart systemd-sysd.service
No containers need to be restarted.
User sessions running outdated binaries:
avb23 @ session #1: login[1343]
avb23 @ user manager service: systemd[1490]
No VM guests are running outdated hypervisor (qemu) binaries on this host.
avb23@avb:~$
```

Identificación de las interfaces de red

1. Escribamos en la terminal el siguiente comando:
 - `ifconfig`
2. Veremos que nos aparece un bloque de texto con información:
 - **lo**
 - **Nuestra interfaz principal.** Aquí es donde vemos nuestra IP.

```
avb23@avb:~$ ifconfig
ens33: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.192.138 netmask 255.255.255.0 broadcast 192.168.192.255
    inet6 fe80::20c:29ff:feae:6ca2 prefixlen 64 scopeid 0x20<link>
    ether 00:0c:29:ae:6c:a2 txqueuelen 1000 (Ethernet)
    RX packets 64 bytes 24601 (24.6 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 63 bytes 7206 (7.2 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 112 bytes 8856 (8.8 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 112 bytes 8856 (8.8 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

avb23@avb:~$ _
```

Comprobación de la tabla de enrutamiento

Ahora que sabemos nuestra dirección IP, el siguiente paso lógico en la configuración TCP/IP es conocer cuál es nuestra "puerta de salida" hacia otras redes o hacia Internet. A esto se le llama **Puerta de enlace** (o *Gateway*).

Para descubrirlo, vamos a utilizar otro comando muy útil del paquete *net-tools* que acabamos de instalar.

En nuestra terminal, escribimos el siguiente comando:

- `route -n`

```
avb23@avb:~$ route -n
Kernel IP routing table
Destination Gateway Genmask Flags Metric Ref Use Iface
0.0.0.0 192.168.192.2 0.0.0.0 UG 100 0 0 ens33
192.168.192.0 0.0.0.0 255.255.255.0 U 100 0 0 ens33
192.168.192.2 0.0.0.0 255.255.255.255 UH 100 0 0 ens33
avb23@avb:~$ _
```

Modificación temporal de la dirección IP

Para asignar una nueva IP a nuestra interfaz (vamos a utilizar la IP 10.0.0.1 en la interfaz ens33), escribimos en la terminal:

- `sudo ifconfig ens33 10.0.0.1 netmask 255.255.255.0`

Comprobamos que se ha cambiado correctamente volviendo a ejecutar el comando de visualización:

- `ifconfig`

```
avb23@avb:~$ sudo ifconfig ens33 10.0.0.1 netmask 255.255.255.0
[sudo] password for avb23:
avb23@avb:~$ ifconfig
ens33: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 10.0.0.1 netmask 255.255.255.0 broadcast 10.0.0.255
    inet6 fe80::20c:29ff:feae:6ca2 prefixlen 64 scopeid 0x20<link>
    ether 00:0c:29:ae:6c:a2 txqueuelen 1000 (Ethernet)
    RX packets 1371 bytes 1934134 (1.9 MB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 410 bytes 30175 (30.1 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 124 bytes 9976 (9.9 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 124 bytes 9976 (9.9 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

avb23@avb:~$ _
```

Veremos que nuestra IP ha cambiado. Para deshacer esto, podemos dejar la interfaz inactiva (apagarla) mediante el siguiente comando:

- `sudo ifconfig ens33 down`

Modificación temporal de la tabla de enrutamiento

Al igual que hemos hecho con la IP, también podemos añadir rutas temporales. Vamos a añadir una ruta por defecto (default gateway) simulada para ver cómo se refleja en nuestra tabla de enrutamiento.

Escribimos el siguiente comando:

- `sudo route add default gw 10.0.0.2`

Comprobamos que la ruta se ha añadido correctamente mirando de nuevo nuestra tabla:

- `route -n`

```
avb23@avb:~$ sudo route add default gw 10.0.0.2
avb23@avb:~$ route -n
Kernel IP routing table
Destination Gateway Genmask Flags Metric Ref Use Iface
0.0.0.0 10.0.0.2 0.0.0.0 UG 0 0 0 ens33
0.0.0.0 192.168.192.2 0.0.0.0 UG 100 0 0 ens33
10.0.0.0 0.0.0.0 255.255.255.0 U 100 0 0 ens33
192.168.192.0 0.0.0.0 255.255.255.0 U 100 0 0 ens33
192.168.192.2 0.0.0.0 255.255.255.255 UH 100 0 0 ens33
avb23@avb:~$
```

Como esto es solo una prueba y no queremos dejar basura en nuestra configuración, vamos a eliminar esa ruta que acabamos de crear:

- `sudo route del default gw 10.0.0.2`

Configuración persistente de la red (Netplan)

Como estamos utilizando **Ubuntu Server 24**, la forma moderna y correcta de hacer que nuestra configuración de red se guarde de forma permanente (y no se borre al reiniciar) es mediante una herramienta llamada **Netplan**, en lugar del antiguo fichero `/etc/network/interfaces` que se usaba en versiones viejas.

Primero, vamos a buscar el archivo de configuración. Nos movemos a la carpeta de Netplan y listamos lo que hay dentro:

- `cd /etc/netplan/`
- `ls`

```
avb23@avb:/etc/netplan$ ls
50-cloud-init.yaml
```

Veremos un archivo con extensión `.yaml`. Vamos a editarlo con permisos de administrador usando el editor nano:

- `sudo nano 50-cloud-init.yaml`
- Dentro del archivo, modificamos el texto para poner nuestra IP estática. Lo dejamos con esta estructura:

network:

version: 2

ethernets:

ens33:

dhcp4: no

addresses: [10.0.0.10/24]

routes:

- to: default

via: 10.0.0.1

nameservers:

addresses: [8.8.8.8, 8.8.4.4]

Finalmente, aplicamos esta configuración para que el sistema la asuma:

- `sudo netplan apply`

```
network:
  version: 2
  ethernets:
    ens33:
      dhcp4: no
      addresses: [10.0.0.10/24]
      routes:
        - to: default
          via: 10.0.0.1
      nameservers:
        addresses: [8.8.8.8, 8.8.4.4]

avb230avb:/etc/netplan$ sudo netplan apply
avb230avb:/etc/netplan$
```

Gestión de la caché ARP

Esta caché se encarga de asociar direcciones lógicas (IP) con direcciones físicas (MAC Ethernet). Primero, consultamos la caché actual de nuestra máquina ejecutando la siguiente orden:

- `arp -a`

También probamos a añadir una entrada manual (estática) en la tabla ARP. Para ello, inventamos una IP y una dirección MAC falsas y utilizamos la opción -s:

- `sudo arp -s 10.0.0.2 01:02:03:04:05:06`

Volvemos a comprobar con `arp -a` y veremos que se ha añadido con la etiqueta "PERM" (permanente). Para no dejar esta configuración falsa (basura) en nuestro sistema, procedemos a borrarla con la opción -d:

- `sudo arp -d 10.0.0.2`

```
avb23@avb:~$ arp -a
? (10.0.0.1) at <incomplete> on ens33
avb23@avb:~$ sudo arp -s 10.0.0.2 01:02:03:04:05:06
[sudo] password for avb23:
avb23@avb:~$ arp -a
? (10.0.0.2) at 01:02:03:04:05:06 [ether] PERM on ens33
? (10.0.0.1) at <incomplete> on ens33
avb23@avb:~$ sudo arp -d 10.0.0.2
avb23@avb:~$
```

Comprobación de la conectividad: Comando Ping

Por defecto en Linux, este comando se ejecuta indefinidamente (habría que pararlo con Ctrl+C). Para evitarlo, ensayamos utilizando la opción -c para indicarle que envíe un número exacto de paquetes (por ejemplo, 4):

- `ping -c 4 8.8.8.8`

Una vez termina, observamos detenidamente el resumen estadístico que nos arroja el sistema, prestando especial atención al porcentaje de paquetes perdidos y a los tiempos de respuesta mínimo, medio y máximo

```
avb23@avb:~$ ping -c 4 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data:
64 bytes from 8.8.8.8: icmp_seq=1 ttl=128 time=5.00 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=128 time=4.57 ms
64 bytes from 8.8.8.8: icmp_seq=3 ttl=128 time=4.77 ms
64 bytes from 8.8.8.8: icmp_seq=4 ttl=128 time=5.98 ms

--- 8.8.8.8 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3006ms
rtt min/avg/max/mdev = 4.573/5.080/5.982/0.542 ms
avb23@avb:~$
```

Captura de tráfico de red: tcpdump

Primero, hacemos una captura rápida que nos muestre los paquetes directamente por el terminal (recuerda cambiar ens33 por el nombre real de tu interfaz):

- `sudo tcpdump -i ens33 -c 5` (Añadimos `-c 5` para que solo capture 5 paquetes y termine automáticamente, en vez de quedarse colgado).

Seguidamente, y tal como requiere el guion, realizamos una captura más profesional guardando el tráfico en un archivo, para poder analizarlo más tarde.

Usamos el parámetro `-w` seguido del nombre del archivo:

- `sudo tcpdump -i ens33 -w captura_trafico.cap -c 10`

```
avb23@avb:~$ sudo tcpdump -i ens33 -c 5
[sudo] password for avb23:
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on ens33, link-type EN10MB (Ethernet), snapshot length 262144 bytes
11:02:42.617465 IP avb.57888 > alphyn.canonical.com.ntp: NTPv4, Client, length 48
11:02:42.646604 IP avb.50716 > _gateway.domain: 16060+ [1au] PTR? 157.91.189.91.in-addr.arpa. (55)
11:02:42.725118 IP alphyn.canonical.com.ntp > avb.57888: NTPv4, Server, length 48
11:02:43.263170 IP _gateway.domain > avb.50716: 16060 1/0/1 PTR alphyn.canonical.com. (89)
11:02:43.264187 IP avb.42422 > _gateway.domain: 43543+ [1au] PTR? 139.192.168.192.in-addr.arpa. (57)
5 packets captured
0 packets received by filter
0 packets dropped by kernel
avb23@avb:~$ sudo tcpdump -i ens33 -w captura_trafico.cap -c 10
tcpdump: listening on ens33, link-type EN10MB (Ethernet), snapshot length 262144 bytes
10 packets captured
10 packets received by filter
0 packets dropped by kernel
avb23@avb:~$ _
```

Auditoría de conexiones con el comando ss

Para concluir la primera parte de la práctica antes de entrar a configurar VLANs, exploramos el comando `ss`, que es la herramienta moderna que sustituye al antiguo `netstat`. En nuestro terminal, ensayamos diferentes variantes de este comando para ver su versatilidad filtrando puertos:

- Listamos todas las conexiones estándar: ss

[illegible]

- Listamos todos los puertos a la escucha y sin escucha: ss -a

```

_w_dgr ESTAB 0 0 * 15082
_w_dgr ESTAB 0 0 * 1843
_w_s1r ESTAB 0 0 * 15185
_w_s1r ESTAB 0 0 * 19391
_w_s1r ESTAB 0 0 * 2876
_w_s1r ESTAB 0 0 * 1879
_w_s1r ESTAB 0 0 * 15159
_w_dgr ESTAB 0 0 * 2771
_w_s1r LISTEN 0 4096 /run/systemd/private 18624
_w_dgr ESTAB 0 0 * 18783
_w_s1r LISTEN 0 4096 * 14784
_w_s1r LISTEN 0 4096 /run/systemd/userd/lo_systemd_dynamicuse 18628
_w_s1r LISTEN 0 4096 /run/systemd/lo_systemd_managedoom 18630
_w_dgr ESTAB 0 0 * 14826
_w_s1r ESTAB 0 0 * 14854
_w_s1r LISTEN 0 4096 /run/lvm/lvmold.socket 15390
_w_dgr ESTAB 0 * 1541
_w_dgr UNKNOWN 0 0 /run/systemd/journal/syslog 15401
_w_s1r LISTEN 0 4096 /run/systemd/fsc.progress 15403
_w_dgr ESTAB 0 0 /run/systemd/journal-dex.log 15405
_w_s1r ESTAB 0 0 /run/systemd/journal/socket 15407
_w_s1r LISTEN 0 4096 /run/systemd/journal/stdout 15409
_w_seq ESTAB 0 4096 /run/userd/control 15413
_w_dgr ESTAB 0 0 * 2770
_w_s1r ESTAB 0 0 /run/systemd/journal/stdout 7865
_w_dgr ESTAB 0 0 * 2889
_w_s1r LISTEN 0 4096 /run/dbus/systemd/socket 5856
_w_dgr ESTAB 0 0 * 1534
_w_s1r LISTEN 0 4096 /run/ldd-installer.socket 4030
_w_s1r LISTEN 0 4096 /run/snmpd.socket 4032
_w_s1r LISTEN 0 4096 /run/snmpd-snmp.socket 4033
_w_s1r LISTEN 0 4096 /run/uuid/request 4034
_w_s1r ESTAB 0 0 * 1568
_w_s1r LISTEN 0 0 /org.kernel/linux.storage/multipath 15740
_w_s1r ESTAB 0 0 05a51eb1ab055570:/bus/systemd-livenessd-bus-api-livenessd 15406
_w_s1r ESTAB 0 0 6c3cc95021d99c27:/bus/systemd-resolve-bus-api-resolve 15185
_w_s1r ESTAB 0 0 61152a3a20b9f61d:/bus/systemd-bus-api-system 17864
_w_s1r ESTAB 0 0 48497c7ad01b687c:/bus/systemd-bus-system 15254
_w_s1r ESTAB 0 0 01731e637b5a957:/bus/systemd-logind-system 20055
_w_s1r ESTAB 0 0 deccdb54b65a2601:/bus/systemd-bus-api-network 15893
_w_s1r LISTEN 0 4096 1B535HWD_ABSTRACT_NETWORK 4069
_w_s1r LISTEN 0 4096 1B535HWD_ABSTRACT_NETWORK 4069
_wdp UNKNOWN 0 0 127.0.0.54:domain 127.0.0.54:domain
_wdp UNKNOWN 0 0 127.0.0.53:domain 127.0.0.53:domain
_wdp UNKNOWN 0 0 192.168.192.195:netbios-ns 192.168.192.195:netbios-ns
_wdp LISTEN 0 4096 127.0.0.54:domain 127.0.0.54:domain
_wdp LISTEN 0 4096 127.0.0.53:domain 127.0.0.53:domain
_w_s1r ESTAB 0 0 447926151:1023

```

- Filtramos para ver solo las conexiones TCP: ss -t

```
avb23@avb:~$ ss -t
State      Recv-Q      Send-Q      Local Address:Port      Peer Address:Port      Process
avb23@avb:~$
```

- Filtramos para ver solo las conexiones UDP: ss -ua

```
avb23@avb:~$ ss -ua
State      Recv-Q      Send-Q      Local Address:Port      Peer Address:Port      Process
UNCONN     0            0            127.0.0.54:domain      0.0.0.0:*
UNCONN     0            0            127.0.0.53:domain      0.0.0.0:*
UNCONN     0            0            192.168.192.199:ens33:bootpc 0.0.0.0:*
avb23@avb:~$
```

- Sacamos un resumen de estadísticas del sistema: ss -s

```
avb23@avb:~$ ss -s
Total: 147
TCP: 2 (estab 0, closed 0, orphaned 0, timewait 0)

Transport Total      IP      IPv6
RAW        1          0        1
UDP        3          3         0
TCP        2          2         0
INET       6          5         1
FRAG       0          0         0

avb23@avb:~$
```

Gestión de VLAN en Linux

Preparación del entorno virtual en VMware

El primer paso fundamental es asegurarnos de que ambas máquinas estén conectadas físicamente al mismo "cable virtual" para que puedan verse entre sí sin interferencias del exterior.

Para ello, nos vamos a la configuración de las máquinas en VMware (Clic derecho en la máquina -> **Settings** -> **Network Adapter**) y asignamos ambas a la misma red privada.

Instalación de las herramientas para VLAN

Una vez arrancadas las dos máquinas virtuales y con la red local configurada, lo primero que hacemos en **ambas** es instalar el paquete que gestiona las VLANs. Introducimos los siguientes comandos en las terminales:

- `sudo apt update`
- `sudo apt install vlan`

Carga del módulo 802.1Q

Para que nuestro sistema operativo GNU/Linux sea capaz de entender y etiquetar los paquetes con el estándar de VLAN (802.1Q), necesitamos cargar su módulo correspondiente en el núcleo (kernel) del sistema. Ejecutamos en los dos servidores:

- `sudo modprobe 8021q`

Para comprobar que se ha cargado correctamente en memoria, usamos el comando:

- `lsmod | grep 8021q`

Además, para hacer que este módulo se cargue automáticamente cada vez que encendamos la máquina en el futuro, lo añadimos al archivo de configuración de módulos:

- `echo "8021q" | sudo tee -a /etc/modules`

```

avb1999@avb:~$ sudo modprobe 8021q
avb1999@avb:~$ lsmod | grep 8021q
8021q                45056  0
garp                  20480  1 8021q
mrp                   20480  1 8021q
avb1999@avb:~$ echo "8021q" | sudo tee -a /etc/modules
echo8021q: command not found
avb1999@avb:~$ echo "8021q" | sudo tee -a /etc/modules
8021q
avb1999@avb:~$

```

Configuración de las VLAN mediante Netplan

Como estamos en Ubuntu 24, descartamos el uso del antiguo archivo `/etc/network/interfaces` que se utilizaba en versiones pasadas y procedemos a configurar las interfaces virtuales utilizando **Netplan**.

En el **Servidor 1**, primero comprobamos cómo se llama nuestra tarjeta de red principal ejecutando `ip a`. Luego, abrimos el archivo de configuración:

```

avb23@avb:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:b2:cf:87 brd ff:ff:ff:ff:ff:ff
    altname enp2s1
    inet6 fe80::20c:29ff:feb2:cf87/64 scope link
        valid_lft forever preferred_lft forever
avb23@avb:~$

```

- `sudo nano /etc/netplan/50-cloud-init.yaml`
- Borramos el contenido que tenga y lo dejamos con esta estructura para crear la **VLAN 10**:

network:

version: 2

ethernets:

ens33:

dhcp4: no

vlan:

vlan10:

id: 10

link: ens33

addresses: [192.168.10.1/24]

Guardamos los cambios (Ctrl+O, Enter) y salimos (Ctrl+X). Aplicamos la configuración con:

- `sudo netplan apply`

```
network:
  version: 2
  ethernet:
    ens33:
      dhcp4: no
  vlans:
    vlan10:
      id: 10
      link: ens33
      addresses: [192.168.10.1/24]

avb1999@avb:~$ sudo netplan apply
avb1999@avb:~$
```

En el **Servidor 2**, hacemos exactamente lo mismo, pero le asignamos la IP final .2 para que estén dentro de la misma red lógica. Su archivo de Netplan quedará así:

network:

version: 2

ethernet:

ens33:

dhcp4: no

vlans:

vlan10:

id: 10

link: ens33

addresses: [192.168.10.2/24]

Aplicamos también con sudo netplan apply.

```
network:
  version: 2
  ethernet:
    ens33:
      dhcp4: no
  vlans:
    vlan10:
      id: 10
      link: ens33
      addresses: [192.168.10.2/24]

avb199@avb:~$ sudo netplan apply
avb23@avb:~$
```

Comprobación de las interfaces y pruebas de conectividad

Para verificar que las interfaces virtuales se han creado correctamente, ejecutamos el comando ip a en ambas máquinas. Deberíamos ver nuestra interfaz principal y una nueva subinterfaz llamada vlan10 con su respectiva dirección IP asignada.

```
avb199@avb:~$ sudo netplan apply
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:00:d0:1f brd ff:ff:ff:ff:ff:ff
    altname enp2s1
    inet6 fe80::20c:29ff:fe00:d01f/64 scope link
        valid_lft forever preferred_lft forever
3: vlan10@ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default qlen 1000
    link/ether 00:0c:29:00:d0:1f brd ff:ff:ff:ff:ff:ff
    inet 192.168.10.1/24 brd 192.168.10.255 scope global vlan10
        valid_lft forever preferred_lft forever
    inet6 fe80::20c:29ff:fe00:d01f/64 scope link
        valid_lft forever preferred_lft forever
avb199@avb:~$

avb23@avb:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:b2:cf:07 brd ff:ff:ff:ff:ff:ff
    altname enp2s1
    inet6 fe80::20c:29ff:feb2:cf07/64 scope link
        valid_lft forever preferred_lft forever
3: vlan10@ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default qlen 1000
    link/ether 00:0c:29:b2:cf:07 brd ff:ff:ff:ff:ff:ff
    inet 192.168.10.2/24 brd 192.168.10.255 scope global vlan10
        valid_lft forever preferred_lft forever
    inet6 fe80::20c:29ff:feb2:cf07/64 scope link
        valid_lft forever preferred_lft forever
avb23@avb:~$
```

Para finalizar y demostrar que nuestra configuración ha sido un éxito, realizamos una prueba de conectividad haciendo ping desde el Servidor 1 al Servidor 2 a través de la IP de la VLAN:

- `ping -c 4 192.168.10.2`

Al recibir respuesta de los paquetes, demostramos que la comunicación a través del túnel lógico de la VLAN 10 está funcionando a la perfección.

```
avb1999@avb:~$ ping -c 4 192.168.10.2
PING 192.168.10.2 (192.168.10.2) 56(84) bytes of data.
64 bytes from 192.168.10.2: icmp_seq=1 ttl=64 time=0.742 ms
64 bytes from 192.168.10.2: icmp_seq=2 ttl=64 time=1.67 ms
64 bytes from 192.168.10.2: icmp_seq=3 ttl=64 time=0.230 ms
64 bytes from 192.168.10.2: icmp_seq=4 ttl=64 time=0.811 ms

--- 192.168.10.2 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3046ms
rtt min/avg/max/mdev = 0.230/0.864/1.674/0.518 ms
avb1999@avb:~$
```

Conclusión

A lo largo de este ejercicio hemos logrado varios hitos importantes:

- **Configuración temporal y diagnóstico:** Hemos aprendido a utilizar las herramientas del paquete net-tools e iproute2 para ver y modificar IPs, gestionar la tabla de enrutamiento de forma temporal y auditar puertos con el comando ss.
- **Análisis de red:** Hemos comprendido cómo el sistema asocia las direcciones lógicas y físicas manejando la caché ARP, y hemos capturado tráfico real utilizando tcpdump para su posterior análisis.
- **Configuración persistente:** Nos hemos adaptado a los estándares actuales configurando la red de forma permanente mediante archivos .yaml a través de la herramienta **Netplan** en Ubuntu Server 24.
- **Gestión de VLANs:** Finalmente, hemos conseguido segmentar nuestra red física creando subinterfaces lógicas etiquetadas bajo el estándar **802.1Q**, demostrando con éxito que el tráfico viaja aislado a través de la VLAN 10.

En definitiva, hemos configurado e implementado con éxito una infraestructura de red básica y segmentada en un entorno virtualizado basado en GNU/Linux.